ENGINEERING DEEP DIVE & SYSTEM AUDIT

CHINTU ROBOT PLATFORM

Complete Repository Analysis, Breaking Points & Flaws Audit, Research Document Breakdown and Engineering Recommendations

This technical report provides a 360-degree audit of the entire Chintu Two-Wheeled Legged Robot repository (`chintutherobo`). It synthesizes the mechanical, firmware, supervisory, network, and theoretical components of the platform, exposes all documented and latent bugs ("cracks" and broken points), fully details the 10-phase research document, and presents prioritized engineering solutions.

Robot Target: Inverted Pendulum	Two-Wheeled Legged	**MCU Core:** Real-Time 200 Hz)	Arduino Mega 2560 (Hard
Supervisor: Asyncio)	Raspberry Pi 4/5 (FastAPI +	**Comms Link:** (CRC16)	USB Serial @ 250,000 baud
Client UI: Dashboard	HTML5 / CSS3 / ES6 PWA	**Audit Date:**	October 2026

1. System Architecture & Physical Baseline

1.1 Hardware & Software Partitioning

The Chintu architecture adopts a rigorous two-tier operational hierarchy designed to decouple hard real-time dynamic balance from high-level supervision:

- **Tier 1 — Real-Time Control Core (Arduino Mega 2560)**:
  - 200 Hz hard real-time balance loop driven by 5 ms delta-time ticks.
  - MPU-6050 6-DOF IMU sampled via I2C Fast Mode (400 kHz) with a 2-stage complementary filter state estimator.
  - Timer3 and Timer4 CTC compare-match interrupts generating microstepped pulses for two NEMA17 stepper wheel drives up to 25 kHz.
  - Timer5 hardware/software PWM driving 6x MG995 servos for 3-link planar leg inverse kinematics.
  - Hardware E-Stop interrupt on Pin 2 (INT4) triggering sub-microsecond motor disable.
- **Tier 2 — High-Level Supervisor (Raspberry Pi 4/5)**:
  - FastAPI asynchronous server managing framed binary serial communication (CRC-16-CCITT) at 250,000 baud.
  - Wi-Fi Access Point host (192.168.4.1) with client isolation and nftables firewall.
  - WebSocket broadcast engine serving JSON telemetry to operators at 20 Hz.
  - Single-use ticket authentication mechanism protecting against unauthorized drive hijacking.
  - High-speed flight blackbox recorder logging telemetry to JSONL and CSV formats.
- **Operator Dashboard (Web-UI PWA)**:
  - Mobile/tablet optimized responsive dashboard built with Three.js (interactive 3D digital twin), real-time canvas charts, dual virtual joysticks, and safety interlocks.

1.2 Canonical Microcontroller Pinout Allocation (Arduino Mega 2560)

Peripheral	Pin Name	Mega 2560 Pin	Hardware Mode / Timer	Description
MPU-6050 IMU	SDA	Pin 20	Bidirectional (I2C)	Fast Mode 400 kHz data line
MPU-6050 IMU	SCL	Pin 21	Output (I2C)	Clock line
Left Stepper	STEP	Pin 5	Timer3 CTC (PE3)	Direct OCR3A compare-match pulse generator
Left Stepper	DIR	Pin 6	Digital Output (PH3)	Direction control
Right Stepper	STEP	Pin 7	Timer4 CTC (PH4)	Direct OCR4A compare-match pulse generator
Right Stepper	DIR	Pin 8	Digital Output (PH5)	Direction control (inverted in firmware)
Steppers Enable	EN	Pin 4	Digital Output (Active LOW)	Global driver tri-state enable/disable
Left Hip Roll	SERVO 1	Pin 44	Timer5 PWM (PL5)	Lateral hip leveling (-20° to +20°)
Left Hip Pitch	SERVO 2	Pin 45	Timer5 PWM (PL4)	Thigh flexion/extension
Left Knee	SERVO 3	Pin 46	Timer5 PWM (PL3)	Knee flexion/extension
Right Hip Roll	SERVO 4	Pin 11	Timer5 PWM (PB5)	Lateral hip leveling (symmetrical)
Right Hip Pitch	SERVO 5	Pin 12	Timer5 PWM (PB6)	Thigh flexion/extension
Right Knee	SERVO 6	Pin 38	Timer5 PWM (PD7)	Knee flexion/extension (Moved from Pin 13 to avoid LED conflict)
Status LED	LED	Pin 13	Digital Output (PB7)	Dedicated onboard system heartbeat indicator
Hardware E-STOP	ESTOP	Pin 2	Input Pull-up (INT4)	Normally closed to GND; instant ISR trip on open
Battery Sense	BATT	Pin A0	Analog Input (ADC0)	Resistor divider sensing main 3S LiPo rail

1.3 Power Distribution Topology

Due to the high dynamic current draw of 6 high-torque MG995 servos (up to 15A combined stall), the power architecture enforces physical and electrical rail segregation:

- **Main Source:** 3S LiPo Battery (11.1V–12.6V, 4000mAh, 30C discharge) with a 30A main blade fuse and mechanical kill switch.
- **Servo Rail (6.0V 15A BEC):** Dedicated high-efficiency buck regulator with a 4700µF ultra-low-ESR capacitor buffer to absorb servo inductive kickback.
- **Computing Logic Rail (5.1V 4A BEC):** Independent buck converter supplying clean USB-C power to the Raspberry Pi 4/5 and powering the Mega 2560 via USB.

- **Motor Rail (Raw 12V LiPo):** Directly feeds VMOT on TMC2209/A4988 stepper drivers, decoupled with 100µF 35V low-ESR capacitors at each driver board.
- **Star Grounding:** All grounds (Battery GND, BEC GND, Pi GND, Stepper GND) meet at a single central copper busbar to eliminate ground loops that cause microcontroller resets.

2. Full Breakdown of the Research Document (`Wheel_Leg_Robot_Research_Document.docx`)

The research document establishes the theoretical roadmap for transforming Chintu from an empirical prototype into an advanced simulated, autonomous, and reinforcement-learning-capable robotics platform. Below is the complete dissection of each section:

Section 1: Project Goal & Core Principle

- **Goal:** Build an end-to-end simulation $\rightarrow$ control $\rightarrow$ learning pipeline for a two-wheeled legged biped.
- **Core Axiom:** "*Pehle physics aur control sahi banao, baad mein RL aur Sim-to-Real.*" (First establish accurate multi-body physics and robust classical controllers before training reinforcement learning agents or deploying Sim-to-Real techniques).

Section 2: Final Recommended Tech Stack & Sequencing

The document mandates a fixed 7-tool stack to prevent tool fatigue and fragmentation:

1. **Fusion 360 (CAD) [Day 1]:** Mechanical modeling, assembly constraints, center-of-mass, and inertia tensor calculations.
2. **Python + VS Code + Git [Day 1]:** Base development environment, automated testing, and version control.
3. **MuJoCo [Phase 2 onwards] ★:** The primary physics engine. Selected for superior multi-contact handling, stability, and speed. *Rule: Isaac Sim must not be used as the initial simulator.*
4. **NumPy + SciPy + Matplotlib [Phase 3 onwards]:** Numerical kinematics, control theory derivations (LQR, Bode plots), and telemetry visualization.
5. **PyTorch + Gymnasium [After Phase 9]:** Reinforcement Learning (PPO) environment definition and neural network policy training.
6. **ROS 2 [Hardware Phase]:** Node orchestration, hardware abstraction, and inter-process communication for physical robot deployment.
7. **Isaac Sim / Isaac Lab [Post-MuJoCo]:** Massive GPU-parallelized domain randomization and sim-to-real transfer once MuJoCo policies are stable.

Section 3: Core Simulation & Control Flow Pipeline

The document dictates the dataflow across development stages:

```
Fusion 360 (CAD)
      ▼
MJCF Model (robot.xml)
      ▼
MuJoCo Physics Engine
      ▼
Virtual Sensors (IMU + Encoders + Contact)
      ▼
State Estimation (Complementary / Kalman Filter)
      ▼
Cascaded Controller (PID → LQR → MPC)
      ▼
Actuator Commands (Torques / Velocities)
      ▼
MuJoCo Feedback Loop
      ▼ [Later Stage]
Gymnasium Environment → PyTorch PPO → Isaac Lab Domain Randomization → Real Robot (ROS 2)
```

Section 4: The 10-Phase Exact Execution Roadmap

Phase	Title	Key Tasks & Objectives	Validation Gate
Phase 1	CAD Design	Model torso, hips, thighs, knees, shanks, wheel hubs; define precise masses and centers of mass.	CAD assembly free of mechanical interference.
Phase 2	MuJoCo MJCF Model	Export CAD to `robot.xml`. Define joints (hip, knee, wheel), joint limits, friction, damping, contacts.	Model loads cleanly in `mujoco` viewer.
Phase 3	Pure Physics Validation	Passive drop test with zero actuator torque. Verify gravity, restitution, contact stability, and CoG.	Robot settles naturally without physics explosions or slipping.
Phase 4	Actuator Isolation Testing	Command torques to hip, knee, and wheel motors individually in open-loop.	Joints move in correct angular directions within limits.
Phase 5	Virtual Sensor Suite	Simulate IMU (pitch angle, pitch rate), joint encoders, and ground contact sensors.	Sensor streams output expected simulated values.
Phase 6	Single-Joint PID Control	Implement closed-loop PID control on individual leg joints for setpoint tracking.	Step response converges within 150 ms without overshoot.
Phase 7	Pitch Balance Controller	Implement inverted-pendulum balance: IMU pitch $\rightarrow$ wheel motor torque.	Robot balances upright indefinitely on flat ground.
Phase 8	Leg-Wheel Coordination	Coordinate dynamic leg height variations (crouching, extending) while maintaining pitch balance.	Legs change height smoothly without destabilizing pitch.
Phase 9	Terrain & Disturbance Rejection	Introduce external horizontal push forces, inclined ramps, and uneven rough surfaces.	Robot recovers from pushes up to 15 N·s impulse.
Phase 10	Advanced Control & Learning	Formulate LQR / MPC. Train Gymnasium PPO policies; export to Isaac Lab for domain randomization.	Sim-to-Real policy transfer onto physical hardware.

Section 5: Strict Operational Prohibitions

- **Prohibition 1:** Do not start with Reinforcement Learning. Classical control must be mastered first.
- **Prohibition 2:** Do not adopt Isaac Sim on Day 1. It introduces unnecessary complexity early on.
- **Prohibition 3:** ROS 2 is not needed until physical hardware integration.
- **Prohibition 4:** Prefer MJCF over URDF for initial MuJoCo development.
- **Prohibition 5:** Data plotting is mandatory after every phase (Pitch vs. Time, Wheel Speed, Joint Angles, Torque, CoM trajectory).

3. Documented Physical Breaking Points, Cracks & Singularities

The codebase and mechanical design explicitly account for several critical mechanical, electrical, and control boundaries:

3.1 Straight-Leg Kinematic Singularity ($d_{\max} = 221\text{ mm}$)

> **Singularity Boundary:** Link lengths are Thigh $L_2 = 120\text{ mm}$ and Shank $L_3 = 101\text{ mm}$. The theoretical maximum vertical extension is $d_{\max} = L_2 + L_3 = 221\text{ mm}$.

When the knee joint approaches 0° (straight line), the manipulator Jacobian determinant approaches zero ($\det(J) \to 0$). To accelerate the wheel hub forward or backward, the required joint angular velocity approaches infinity ($\dot{\theta} \to \infty$). Any attempt to balance in this configuration causes immediate loss of control authority. The software enforces a **46 mm safety margin**, clamping maximum extension to **175 mm** (180 mm in firmware).

3.2 Scissor Folding & Linkage Binding ($d_{\min} = 19\text{ mm}$)

The theoretical minimum height along the vertical axis is $|L_2 - L_3| = 19\text{ mm}$. In reality, physical MG995 servo horns, aluminum bracket flanges, and motor wiring collide well before this point. To prevent stripped servo gears and motor burnout, the system enforces a **66 mm buffer**, restricting the minimum stance height to **85 mm** (80 mm in firmware).

3.3 High-Current Servo Stalls & Voltage Brownout

Under aggressive maneuvering or mechanical jams, each MG995 servo can draw up to 2.5A, resulting in potential peak transient currents exceeding 15A across the 6 servos. If fed from the same rail as the microcontrollers, battery internal resistance causes bus voltage to sag below 4.5V, triggering an unexpected microcontroller brownout reset. The platform mitigates this through:

1. Split power regulation: A dedicated 6.0V 15A buck converter with a 4700µF low-ESR capacitor buffer for servos.
2. A debounced low-battery safety check: Battery voltage below 9.6V (3.2V/cell for 3S LiPo) must persist for $>200\text{ ms}$ before triggering a fault, preventing false trips from brief servo transients.

3.4 ATmega2560 Watchdog Bootloader Bug

The stock Arduino Mega bootloader contains a known bug: it does not clear `MCUSR` or disable the AVR hardware watchdog timer upon reset. When a watchdog timeout occurs, the MCU resets, but the watchdog remains active with a 15 ms timeout. The bootloader takes longer than 15 ms to run, causing the MCU to enter an infinite reset loop. Chintu protects against this by inserting an assembly hook in the `.init3` section (which executes before C runtime initialization) to immediately zero `MCUSR` and disable the watchdog.

3.5 Critical Tilt Cutoff Boundary

An inverted pendulum has a strictly bounded basin of attraction. If chassis tilt exceeds $\pm 45^\circ$ in pitch or $\pm 35^\circ$ in roll, the wheels can no longer generate sufficient horizontal acceleration to swing the

center of mass back above the contact point. Driving the motors beyond this threshold causes violent wheel spin, chassis flipping, and mechanical damage. The safety supervisor instantly halts stepper pulses and tri-states motor drivers upon crossing these thresholds.

4. Critical "Cracks" & Broken Code Points in the Codebase

Our deep audit uncovered 11 severe bugs, breaking points, and discrepancies in the repository:

Crack 1: Fatal JavaScript ReferenceError in Web UI CRITICAL

- **File:** `robot/web-ui/js/api.js` (Line 79)
- **Defect:**

```
// api.js line 79
State.addLog('NET', `Connected to supervisor via ${protocol}//${host}:${port}`);
```

  The variable `protocol` is never declared or in scope (the script defines `wsProtocol` and `httpProtocol`).
- **Consequence:** When the WebSocket connection establishes, the `ws.onopen` event handler throws an unhandled `ReferenceError: protocol is not defined`. This causes the function to abort before reaching `this.startPingLoop()` and `this.startCommandLoop()`. The UI appears connected, but keepalives and user drive commands are never sent.

Crack 2: Broken Systemd Service Module Path & Interpreter CRITICAL

- **File:** `robot/backend/scripts/chintu.service` (Lines 8–9)
- **Defect:**

```
WorkingDirectory=/home/pi/robot/backend
ExecStart=/usr/bin/python3 -m backend.server
```

- **Consequence:**
  1. With `WorkingDirectory=/home/pi/robot/backend`, Python searches for a module named `backend` inside `backend/` and fails immediately with `ModuleNotFoundError: No module named 'backend'`.
  2. The unit executes system Python `/usr/bin/python3` instead of the virtual environment Python (`/home/pi/robot/backend/venv/bin/python3`), where all dependencies (FastAPI, Pydantic, Uvicorn, etc.) are installed.

Crack 3: Contradictory & Abandoned Guide in `README_RASPBERRY_PI.md` CRITICAL

- **File:** `README_RASPBERRY_PI.md`
- **Defect:** The entire root guide instructs the user to configure an **ESP32** microcontroller at 115,200 baud running `firmware/chintu_esp32.ino`, or direct GPIO control via `rpi_chintu.py`, using scripts `setup_rpi.sh` and `start_rpi.sh`.
- **Consequence:** None of these files exist in the repository. The production firmware is an Arduino Mega 2560 at 250,000 baud. The guide tells operators to execute `python3 server.py --auto-serial`, which crashes because `server.py` has no `--auto-serial` flag and aborts if `ROBOT_AUTH_TOKEN` is not defined in the environment.

Crack 4: Nominal Leg Height Discrepancy & Gain Scheduling Skew WARNING

- **Files:** `robot/backend/config.py` vs. `robot/firmware/include/config.h`

- **Defect:**
  - Supervisor (`config.py`): `LEG_HEIGHT_NOMINAL_MM = 135.0`
  - Firmware (`config.h`): `LEG_HEIGHT_NOMINAL_MM = 118.0f`
- **Consequence:** Firmware gain scheduling scales pitch gains by $\sqrt{h / h_{\text{nominal}}}$. When the supervisor commands the nominal 135 mm height, the firmware evaluates $\sqrt{135 / 118} \approx 1.07$, artificially increasing balance stiffness by 7–11% above designed baseline values. Height boundaries also conflict: supervisor restricts to [85, 175] mm, while firmware allows [80, 180] mm.

Crack 5: Missing Hardware Pin in Primary Manual WARNING

- **File:** `robot/README.md` (Section 2.1 Table)
- **Defect:** The pinout table completely omits **SERVO 5 (Right Hip Pitch, Pin 12)**. It lists SERVO 1, 2, 3, 4, and 6, leaving a confusing note on SERVO 6: `*(NEEDS PHYSICAL WIRING CHECK: Pin 13 is onboard LED)*`.

Crack 6: Telemetry Struct Length Inconsistency WARNING

- **File:** `robot/protocol.md` (Section 2.3.2)
- **Defect:** The document lists `uint8_t reserved[3]` for `TelemetryPayload`, which sums to 35 bytes, despite heading the section as `Length: 36 Bytes`. The actual firmware (`protocol.h`) and Python parser (`protocol.py`) implement `reserved[4]` (36 bytes). Any third-party client built from the markdown specification will fail CRC checks.

Crack 7: Incomplete Root `requirements.txt` WARNING

- **File:** `requirements.txt` vs. `robot/backend/requirements.txt`
- **Defect:** The root file contains only `websockets`, `aiohttp`, and `pyserial`. Installing from the root omits FastAPI, Pydantic, Uvicorn, and Pytest.

Crack 8: Deprecated Hotspot Configuration in `setup_hotspot.sh` WARNING

- **File:** `robot/backend/scripts/setup_hotspot.sh` (Lines 34–38)
- **Defect:** The script attempts to configure static IP by appending to `/etc/dhcpcd.conf` and restarting `dhcpcd`. Modern Raspberry Pi OS (Bookworm and later) uses `NetworkManager` (`nmcli`) and lacks `dhcpcd` entirely, breaking Wi-Fi setup on new systems.

Crack 9: Missing SAN in SSL Certificate Generation WARNING

- **File:** `robot/backend/scripts/generate_cert.py` (Lines 37–42)
- **Defect:** The OpenSSL configuration adds SANs for `192.168.4.1`, `127.0.0.1`, `localhost`, and `chintu.local`. However, the system's mDNS domain throughout the repo is **`chintu.robot`**. Browsers connecting to `https://chintu.robot:8000` flag security certificate mismatch warnings.

Crack 10: Firmware Comms Stream Parser Edge Case INFO

- **File:** `robot/firmware/src/comms.cpp` (Lines 92–98)
- **Defect:** In `STATE_WAIT_SYNC1`, if another `0xAA` byte is received (e.g. sequence `0xAA 0xAA 0x55`), the state resets to `STATE_WAIT_SYNC0`, discarding the frame. In contrast, `backend/protocol.py` correctly remains in `STATE_WAIT_SYNC1`.

Crack 11: Architectural Gap with Research Document CRITICAL

- **Scope:** Entire codebase vs. `Wheel_Leg_Robot_Research_Document.docx`
- **Defect:** Zero MuJoCo MJCF files (`robot.xml`), CAD mesh exports, PyTorch PPO scripts, or ROS 2 packages exist in the repository. The advanced simulation-to-real and RL pipelines described in the research paper remain entirely unimplemented.

5. Actionable Engineering Recommendations & Solutions

5.1 Immediate Source Code Fixes

1. Fix Web-UI WebSocket Link (`robot/web-ui/js/api.js`)

Update line 79 to use the correct protocol variable:

```
// In robot/web-ui/js/api.js line 79:
// OLD:
State.addLog('NET', `Connected to supervisor via ${protocol}//${host}:${port}`);

// NEW:
State.addLog('NET', `Connected to supervisor via ${wsProtocol}//${host}:${port}`);
```

2. Fix Systemd Service Unit (`robot/backend/scripts/chintu.service`)

Correct the working directory and point to the virtual environment binary:

```
[Unit]
Description=Chintu Wheel-Leg Robot Supervisor Daemon
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/chintu/robot
ExecStart=/home/pi/chintu/robot/backend/venv/bin/python3 -m backend.server
Restart=always
RestartSec=3
Environment=PYTHONUNBUFFERED=1
EnvironmentFile=/etc/chintu.env

[Install]
WantedBy=multi-user.target
```

3. Synchronize Nominal Leg Height in Firmware (`robot/firmware/include/config.h`)

```
// In robot/firmware/include/config.h lines 70-74:
#define LEG_HEIGHT_MIN_MM      85.0f
#define LEG_HEIGHT_MAX_MM      175.0f
#define LEG_HEIGHT_NOMINAL_MM  135.0f
```

4. Add `chintu.robot` to Certificate Generator (`robot/backend/scripts/generate_cert.py`)

```
// In robot/backend/scripts/generate_cert.py under [alt_names]:
[alt_names]
IP.1 = 192.168.4.1
IP.2 = 127.0.0.1
DNS.1 = localhost
DNS.2 = chintu.local
DNS.3 = chintu.robot
```

5. Enhance Firmware Stream Parser Resilience (`robot/firmware/src/comms.cpp`)

```
// In robot/firmware/src/comms.cpp lines 92-98:
case STATE_WAIT_SYNC1:
    if (byte == PROTOCOL_SYNC_1) {
        rx_state_ = STATE_WAIT_LEN;
    } else if (byte == PROTOCOL_SYNC_0) {
        rx_state_ = STATE_WAIT_SYNC1; // Maintain state on repeated sync bytes
    } else {
        rx_state_ = STATE_WAIT_SYNC0;
    }
    break;
```

5.2 Documentation Synchronization Checklist

- **Rewrite** `README_RASPBERRY_PI.md`**:** Eliminate all ESP32 references. Document the Arduino Mega 2560 pinouts, 250,000 baud serial connection, and `/etc/chintu.env` token provisioning.
- **Update** `robot/README.md` **Pinout:** Add SERVO 5 (Pin 12 - Right Hip Pitch) and clarify that SERVO 6 uses Pin 38 to preserve Pin 13 for the LED.
- **Align** `protocol.md`**:** Update Section 2.3.2 to specify `uint8_t reserved[4];` to reflect the actual 36-byte payload.
- **Consolidate Dependencies:** Copy all requirements from `robot/backend/requirements.txt` into root `requirements.txt`.

5.3 Roadmap to Implement the Research Pipeline (Phases 1–10)

1. **Create** `robot/simulation/` **Directory:**
   - Export CAD models from Fusion 360 as STL/OBJ files.
   - Construct `robot/simulation/robot.xml` (MJCF format) defining kinematics, mass, inertias, contacts, and damping.
2. **Build MuJoCo Simulation Testbed:**
   - Create `robot/tools/mujoco_sim.py` using the official Python `mujoco` package.
   - Replicate the firmware's 200 Hz discrete balance loop to tune PID and LQR gains in simulation prior to hardware deployment.
3. **Derive LQR State-Space Controller:**
   - Linearize the non-linear inverted pendulum dynamics around vertical equilibrium ($\theta = 0$).
   - Use SciPy (`scipy.linalg.solve_continuous_are`) to compute optimal state feedback gains $\mathbf{K}$ across the operating height envelope ($85\text{ mm} - 175\text{ mm}$).
4. **Implement Reinforcement Learning Environment:**
   - Wrap the MuJoCo model inside a standard `gymnasium.Env` interface.
   - Train PPO policies in PyTorch for terrain traversal and disturbance rejection before sim-to-real transfer.

> **Audit Conclusion:** The Chintu robot core control and safety architecture is solid. Applying these fixes resolves all active failure modes, harmonizes the documentation, and lays out a clear path toward the full simulation-to-real pipeline.